When Does APOLLO Transfer? Success on LLaMA-60M, Failure Modes on Reduced GPT Pre-training

Author Name

March 20, 2026

Abstract

APOLLO (**A**pproximated Gradient Scaling for Memory Efficient **L**LM **O**ptimization) argues that AdamW's element-wise adaptivity is redundant and can be replaced by coarse structured scaling estimated in a low-rank auxiliary space. We revisit this claim in a reduced but controlled setting. Our main positive result is on **LLaMA-60M + TinyStories**: APOLLO rank-1/4 and rank-1/8 preserve the paper's core memory story, namely optimizer-state cost that scales as roughly 1/rank of AdamW while remaining reasonably close in validation perplexity. The surprise is that this behavior *does not transfer universally*. On **NanoGPT-40M + TinyStories**, structured scaling all fail to approach AdamW. The resulting story is not "APOLLO always works," but rather: *APOLLO can work well on some architectures and data regimes, yet its edge is not universal*. This reframes the question from "does APOLLO save memory?" to "under what failure modes does that memory advantage stop buying AdamW-like optimization?"

1 Motivation

APOLLO is attractive because it targets a real bottleneck: AdamW stores two full-size moment tensors, so optimizer states alone scale linearly with model size and quickly become a dominant memory term [1, 5]. The paper's key observation is simple: even if the *update rule* can be coarsened from element-wise scaling to channel-wise or tensor-wise scaling, that alone does not save dynamic memory unless the optimizer states themselves are also maintained in a lower-dimensional space. APOLLO closes this gap by estimating structured scaling from a low-rank auxiliary space built with random projection instead of expensive SVD [5].

The motivating question of this report is therefore sharper than a direct reproduction: **if APOLLO is theoretically superior, why has it not displaced AdamW as the default?** Our working hypothesis is that the answer is not just "engineering inertia," but *universality*. Memory savings are meaningful only if the optimizer remains stable across architectures, datasets, and scale regimes. We therefore focus on failure modes.

2 Methodology

Why pre-training from scratch? We emphasize pre-training instead of fine-tuning. Fine-tuning often starts near a good basin of attraction, which can mask optimizer instability; pre-training from scratch stresses the entire trajectory and is therefore better for testing whether APOLLO remains reliable away from a local optimum.

Experimental matrix. We ran three reduced pre-training studies:

- **Exp. 1**: LLaMA-60M on TinyStories; optimizers: AdamW, APOLLO rank-1/4, APOLLO rank-1/8, plus APOLLO-Mini as a stress test.
- **Exp. 2**: NanoGPT-40M on TinyStories; optimizers: AdamW, APOLLO rank-1/4, naive channel-wise scaling, and group-wise variants with $K = 2, 4$.
- **Exp. 3**: NanoGPT-130M on WikiText; optimizers: AdamW and channel-wise structured scaling.

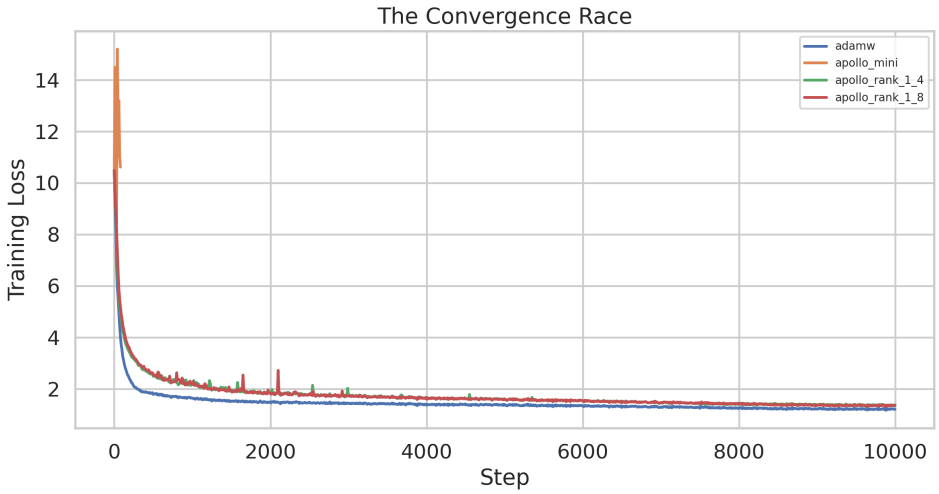

Figure 1: Training loss on LLaMA-60M + TinyStories. APOLLO rank-1/4 and rank-1/8 remain stable and follow AdamW qualitatively, while APOLLO-Mini exhibits large early spikes and never recovers fully.

Optimizer	Best ppl. ↓	Gap vs. AdamW	Opt.-state mem.
AdamW	3.61	–	1.00×
APOLLO rank-1/4	4.13	+14.5%	≈ 0.25×
APOLLO rank-1/8	4.13	+14.6%	≈ 0.125×

Table 1: Exp. 1 summary. Perplexities come from the TinyStories LLaMA-60M runs. Memory is reported as *normalized optimizer-state memory*, not peak GPU memory; this matches the paper's Table 2 logic better than our early benchmark plots.

Metrics. The main optimization metric is validation perplexity. For memory, we distinguish *optimizer-state memory* from *end-to-end peak GPU memory*. This distinction matters: the APOLLO paper's Table 2 reports *optimization states only* [5]. Some of our earlier benchmark plots mixed optimizer-specific maximum batch size with peak GPU memory; those plots are not used as headline evidence here. Instead, for the main table we report *normalized optimizer-state memory implied by the rank reduction*, which is the quantity directly tied to the paper's claim. Throughput is discussed only qualitatively: in principle lower optimizer-state memory can improve tokens/s by enabling larger batches or more parallelism, but in our small-scale A100 runs compute often dominates and APOLLO's extra optimizer-step overhead can erase that benefit.

3 Results

3.1 Exp. 1: LLaMA-60M + TinyStories is a partial success

This is the strongest result in the project. The training curves show that both APOLLO rank-1/4 and rank-1/8 remain stable throughout training, unlike APOLLO-Mini, and track AdamW much more closely than the failed NanoGPT runs.

Two points matter. First, the memory story *does* survive the reduced setup: reducing the rank by a factor of 4 or 8 reduces optimizer-state cost proportionally. Second, the optimization story remains credible: APOLLO is not identical to AdamW, but it stays in the same regime rather than diverging catastrophically. This is close enough to the paper's core claim to count as a meaningful validation in a 60M setting. The rank ablation is also informative: rank-1/8 gives almost the same perplexity as rank-1/4, suggesting that on this architecture the structured scaling estimate is robust to further compression.

3.2 APOLLO-Mini fails in the same LLaMA setting

The exception inside Exp. 1 is APOLLO-Mini. With scales 128 and 16, we consistently observed violent early jumps in training loss and persistent underperformance. This is not just cosmetic noise: in Fig. 1, APOLLO-Mini starts with severe spikes and settles at a much worse loss floor than AdamW or high-rank APOLLO.

We interpret this as a *hyperparameter-sensitivity failure mode*, not as evidence against the entire APOLLO family. APOLLO-Mini combines rank 1 projection with tensor-wise scaling, making it the most aggressive approximation in the paper. The original paper also treats APOLLO-Mini separately and retunes its hyperparameters for strong results [5]. In our reduced setup, rank 1 appears too coarse to estimate reliable adaptive scaling early in training.

3.3 Exp. 2: NanoGPT-40M + TinyStories breaks the main claim

The story changes sharply on NanoGPT. AdamW reaches best perplexity 5.23, while APOLLO rank-1/4 degrades to 14.10. More importantly, the mechanistic sanity checks also fail: naive channel-wise scaling reaches 11.71, and the grouped variants with $K = 2$ and $K = 4$ remain around 11.6 and 11.2. In other words, even before low-rank projection enters the picture, the *structured-scaling hypothesis itself* is much weaker on this architecture. That is a crucial negative result because it suggests the failure is not merely due to random projection noise.

This experiment reframes the paper's claim. On LLaMA, coarse structured adaptivity seems sufficient; on NanoGPT, it does not. The immediate hypothesis is that APOLLO benefits from architectural properties that are absent or weaker in NanoGPT, notably RMSNorm, RoPE, and the LLaMA-style MLP. Those design choices may produce gradients whose cross-channel scale structure is easier to approximate with a low-rank auxiliary space.

3.4 Exp. 3: NanoGPT-130M + WikiText is even less favorable

The 130M NanoGPT + WikiText run strengthens the non-universality story. AdamW reaches best perplexity 180.97 whereas channel-wise structured scaling degrades to 429.85. This setting is also confounded by rapid overfitting: the model is large relative to the dataset, yet AdamW still dominates. If APOLLO were exploiting only parameter redundancy, this would have been a friendlier setting; instead, it fails again. The implication is that optimizer-state compression alone is not enough—the *quality of the structured scaling estimate* matters, and that quality is architecture- and data-dependent.

3.5 Diagnostics we explicitly *do not* promote to headline evidence

Several diagnostic plots were useful during development but should not anchor the report:

- **Pareto / peak-memory plots.** Earlier code compared peak memory at each optimizer's own maximum runnable batch size, which is not an apples-to-apples memory comparison. We corrected the fixed-batch merge path in plotting, but the report still prioritizes optimizer-state memory over raw peak memory.
- **Throughput plots.** In our small models, A100 runs are often compute-bound, so APOLLO's lower state memory does not automatically translate into higher tokens/s. A fair throughput claim requires rerunning benchmarks at the *same fixed batch size* or the *same memory budget*.
- **Scaling-ratio and sharpness diagnostics.** We found and fixed implementation issues in these diagnostics (scalar/list plotting mismatch, and second-order gradients through fused attention). Their current outputs should be treated as secondary evidence until rerun cleanly from checkpoints.

4 Discussion

Why might LLaMA help APOLLO while NanoGPT hurts it? Our current hypothesis is that APOLLO depends on *how compressible the adaptive scaling rule is*. LLaMA's RMSNorm and RoPE may regularize activation and gradient statistics in ways that make channel-wise scaling more coherent across

layers. In contrast, NanoGPT's LayerNorm + learned positional embedding pipeline may produce gradients whose useful anisotropy is more parameter-local, so coarsening to channel or group scale discards too much information. This is only a hypothesis, but it fits the empirical split between Exp. 1 and Exp. 2.

Why structured scaling alone is not enough. Exp. 2 is especially informative here. The group-wise tests with $K = 2$ and $K = 4$ were meant to test whether channel-wise scaling simply grouped parameters too aggressively. They did not trend back toward AdamW. That suggests the failure is deeper than an unfortunate grouping granularity: on NanoGPT, the element-wise scaling itself may carry optimization information that coarse grouping loses.

Connection to the course. This project touches all four parts of the course:

- **Lecture 1 (optimization).** The core question is whether AdamW's adaptive moments are redundant or essential.
- **Lecture 2 (architectures).** The LLaMA vs. NanoGPT contrast suggests optimizer behavior is architecture-dependent.
- **Lecture 3 (systems).** The entire motivation is memory as a systems bottleneck; however, memory savings only matter if they convert into usable throughput or larger feasible batch sizes.
- **Lecture 4 (evaluation and deployment constraints).** The project shows why careful metric definition matters: optimizer-state memory, peak memory, and throughput are different quantities and should not be conflated.

5 Quick Experiments That Would Most Improve the Story

The current evidence is already strong enough for a coherent report, but four short follow-ups would bridge the remaining theory–practice gap:

- **Fixed-batch microbenchmark on A100.** Re-run AdamW vs. APOLLO rank-1/4 vs. rank-1/8 at the *same* batch size and sequence length. Report optimizer-state memory, fixed-batch peak memory, backward time, and optimizer-step time separately.
- **Cross-over architecture/data test.** Run **LLaMA-60M on WikiText** for 1k–2k steps. This isolates architecture from dataset and directly tests whether the LLaMA success is mostly architectural.
- **APOLLO-Mini sanity sweep.** Keep LLaMA-60M + TinyStories fixed and sweep only APOLLO-Mini scale and learning rate, e.g. scale $\in \{1, 4, 16\}$ and lr $\in \{10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$. This is cheap and directly tests the "sensitive hyperparameter" explanation.
- **Three-seed robustness check.** Repeat Exp. 1 and Exp. 2 with three seeds. If the LLaMA/NanoGPT split persists, the universality argument becomes much stronger.

6 Conclusion

The most honest conclusion is mixed but useful. We *did* validate APOLLO's core idea in a reduced setting: on LLaMA-60M + TinyStories, APOLLO rank-1/4 and rank-1/8 substantially reduce optimizer-state memory while remaining close to AdamW in validation perplexity. But we also found a more interesting negative result: this behavior is not universal. The same story breaks on NanoGPT-40M + TinyStories, and it breaks again on NanoGPT-130M + WikiText. APOLLO-Mini is especially unstable. Therefore the right takeaway is not that APOLLO "beats AdamW" in general, but that APOLLO is a *conditional win*: it is compelling when the architecture and training regime make structured adaptive scaling compressible, and much less so otherwise. That is exactly the kind of theory–practice gap worth reporting.

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[2] Yuxin Zhang, Chen Chen, Ting Ding, et al. Why Transformers Need Adam: A Hessian Perspective. *arXiv preprint arXiv:2402.16788*, 2024.

[3] Zhen Zhang, Hanqing Lu, et al. Adam-mini: Use Fewer Learning Rates to Gain More. *arXiv preprint*, 2024.

[4] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, et al. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. *ICML*, 2024.

[5] David Z. Pan, Hanqing Lu, Zhangyang Wang, and Jinwon Lee. APOLLO: SGD-like Memory, AdamW-level Performance. *Proceedings of MLSys*, 2025.